A Verifiable Secure Aggregation for Federated Learning with Low-quality Data

[a]*School of Computer and Software Engineering, Xihua University, Chengdu 610039, China*

Abstract

Federated learning lets multiple participants build a shared model collaboratively while their raw data remain entirely local, and it has shown great potential in various fields. Many studies focus on the security of federated learning to prevent incorrect results return and user data leakage. However, low-quality data are overlooked, which may render the trained global model unusable. To fill this gap, we devise a privacy-preserving and verifiable weighted-aggregation protocol that alleviates the adverse influence of low-quality data on the global model. In the scheme, users' gradients and the weights of gradients are protected by using secret sharing and truth discovery techniques respectively. Meanwhile, the aggregation results computed by the cloud server can be verified. Finally, we conduct simulation experiments on our scheme to demonstrate its feasibility.

Keywords: Federated Learning, Secure Aggregation, Low-quality Data, Verifiable Computation.

1. Introduction

As data-driven applications proliferate, safeguarding personal information has become increasingly imperative. With the continuous collection and processing of personally sensitive information, the risk of data breaches has risen substantially. In response, governments and regulatory organizations have introduced stringent data privacy regulations. For instance, the European Union's General Data Protection Regulation (GDPR) requires minimizing personal data processing and strictly prohibits unauthorized usage

*Corresponding author

[1]. These regulatory initiatives have exerted a profound influence on traditional machine learning methods. Traditional machine learning requires merging large datasets into centralized servers to leverage the data's potential. This centralized paradigm enables predictive models by exploiting comprehensive datasets, enhancing model performance and accuracy. However, this methodology poses challenges when dealing with sensitive data such as healthcare records, financial information, or personal identifiers. In 2016, Google proposed federated learning [2] to address these issues. Federated learning trains models collaboratively on edge devices, transmitting only parameter updates—such as gradients or weightswhile raw data stay local. By decentralising computation, federated learning both preserves confidentiality and scales efficiently over heterogeneous data sources.

Despite federated learning's privacy advantages, researchers have shown that exchanged gradients can still leak information through membership-inference [3] and model-inversion attacks [4]. On the other hand, data quality varies among users in real-life scenarios: high-end device users usually generate high-quality data, while low-performing device users produce low-quality data. Here "low-quality data" means non-adversarial imperfections in a client's local dataset that reduce utility, such as label noise, input perturbations, class imbalance with heterogeneous client sizes, and partial occlusion. Clients remain honest but curious and follow the protocol. Studies [5–7] indicate that low-quality data can slow model convergence or render the model useless during training. In addition, a malicious server might cut corners during aggregation or deliberately falsify results, thereby skewing the learned model [8].

In this paper, we focus on the following question: *How can we construct a federated learning scheme that can effectively protect users' privacy, efficiently utilize data (including low-quality data), and verify the correctness of the aggregated results generated by the cloud server?* Therefore, we propose a verifiable secure weighted aggregation federated learning scheme. The proposed scheme has three properties: (i) mitigates the impact of unreliable users by assigning higher weights to high-quality data, (ii) employs masking to protect both the weighted gradients and the corresponding weight values, and (iii) leverages the additive homomorphism of secret sharing to verify the aggregation results.

The protocol matches deployments where privacy, untrusted aggregation, and heterogeneous data coexist, such as mobile keyboard prediction at scale, multi-institution medical imaging, and financial analytics. These settings

have already applied federated learning in practice, and our single-server, verifiable, reliability-weighted design aligns with such constraints.

Our contributions can be outlined as follows:

- We address low-quality data in cross-device federated learning. The scheme assigns larger weights to higher-quality updates and reduces the impact of non-adversarial defects while individual updates remain hidden.
- We design a verifiable secure aggregation protocol using double masking and Shamir secret sharing with per-round tags. Each user can check the returned sum, the computation is exact over the field, and the protocol tolerates drop-out with a threshold t.
- We implement the protocol and evaluate it on image classification under cross-device constraints. The design uses a single coordination server, tolerates intermittent clients with threshold drop-out, and adds only lightweight per-round checks, keeping the communication cost comparable to single-server baselines.

The rest of the paper is structured as follows: Section 2 reviews related work; Section 3 outlines preliminaries; Section 4 formalizes the problem statement; Section 5 details our method; Section 6 presents security analyses. Section 7 reports the experimental evaluation; and Section 8 concludes.

2. Related Work

2.1. Verifiable Federated Learning

Prior work studies how clients can check the correctness of aggregation while keeping updates hidden. VerifyNet secures gradients with masking and homomorphic hashes and lets clients verify returned results, but it adds extra cryptographic tags and computation and it does not consider data quality or weighted aggregation [9]. VeriFL pursues fast and communication-efficient verification by using linearly homomorphic hashing, yet the check focuses on algebraic correctness of the sum and does not address unreliable updates or robustness to poor data [10]. VFL combines Paillier encryption, the Chinese Remainder Theorem, and bilinear aggregate signatures for verifiable training in industrial IoT, which increases implementation complexity and key management overhead and still requires strong synchronization among

participants [11]. SVCA adopts chained aggregation with commitments to ensure verifiability, but the chaining makes each round depend on prior states and the design does not target the impact of low-quality inputs [12]. Other verifiable designs follow similar lines by authenticating linear relations of the aggregate; they improve integrity against a faulty server but they leave the question of unreliable client data largely open [13, 14].

2.2. FL with low-quality data

We first summarize how non adversarial low quality data is commonly simulated so that later experiments follow established practice. For label noise, studies inject symmetric flips at a given rate and asymmetric flips within confusable pairs, and the asymmetric protocol on CIFAR uses pairs such as truck to automobile, bird to airplane, deer to horse, and cat to dog [15]. For input perturbations that reflect sensor or pipeline imperfections, evaluations adopt the corruption benchmark that provides Gaussian noise and Gaussian blur and brightness and contrast changes with prescribed severities [16]. For imbalance and heterogeneous client size, federated evaluations construct long tailed class frequencies and allocate data to clients with Dirichlet non iid partitions so that some clients hold few samples while others hold more [17–19]. For partial occlusion, random erasing removes a random square region with a chosen probability and area ratio and is widely used as a controllable form of missing content [20].

We next review how representative federated methods handle unreliable contributors. SecProbe perturbs the objective with a differential privacy functional mechanism and selects contributors with the differential privacy exponential mechanism so that a participants quality is hidden from others, and it does not introduce an explicit mechanism for dropout tolerance [6]. PPFDL computes truth discovery weights over encrypted updates using additively homomorphic encryption for sums and Yaos garbled circuits for division and related operations, and it assumes two noncolluding servers and discusses robustness when some users drop out [7]. ESFL gives each client a local tag to verify a returned weighted sum while keeping updates protected and it takes weights as given rather than learning data quality aware reweighting [14]. VeriFL achieves verifiable aggregation with a linearly homomorphic hash and commitments, and the verification communication is independent of model dimension, and the protocol composes with secure aggregation [10].

These practices guide our evaluation. We adopt symmetric and asymmetric label noise at stated rates, client side input perturbations with fixed severities from the corruption benchmark, long tailed distributions with Dirichlet partitions and heterogeneous client sizes, and random erasing with specified probabilities and patch sizes. We keep all other training and cryptographic settings unchanged so that measured differences reflect robustness to data quality alone.

2.3. Application-driven deployments

Real-world deployments indicate that production FL must satisfy stricter requirements than benchmark settings, including privacy at scale, partial participation, and tight communication budgets [21]. A production system report for mobile, cross-device FL details a scalable orchestration stack with secure aggregation and tolerance to client drop-outs [21]. In healthcare, a 10-institution medical-imaging study showed that collaborative training can approach centralized performance and generalize to external sites while avoiding raw data sharing [22]. During the COVID-19 pandemic, a 20-hospital federated study trained prognostic models from clinical and imaging data, demonstrating operational feasibility under heterogeneous, unharmonized datasets and strict compliance constraints [23]. Together, these deployments motivate lightweight, verifiable, and reliability-aware protocols that fit practical constraints of cross-device and cross-silo settings.

3. Preliminaries

In this section, federated learning and truth discovery technologies are introduced first. We then give a concise overview of Shamir secret sharing and the Diffie-Hellman key exchange protocol.

3.1. Federated Learning

Federated learning is a distributed machine learning framework that typically involves a cloud server and N users u_i, where each user u_i holds a locally stored private dataset, $i \in \{1, ..., N\}$. In each iteration $T \in \{1, 2, \dots\}$, the cloud server sends the current global gradient $\boldsymbol{x}^{T-1}$ to every participant. User u_i computes the model parameters $\boldsymbol{x}_i^T$ using the Stochastic Gradient Descent (SGD) algorithm on their sensitive dataset and sends the computed gradients back to the cloud server. Using the federated averaging algorithm [24], the cloud server aggregates all users' gradients to obtain the weighted

average gradient $\boldsymbol{x}^T = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i^T$. It repeats this routine—aggregation and broadcast—until the target accuracy is achieved.

The above process assumes that the datasets of all users are of high quality and have similar distributions. However, low-quality data may reduce the convergence speed of the aforementioned model. Next, we will introduce truth discovery technology, which can assign different weights to data of different qualities, thus mitigating the adverse influence of low-quality samples on the final accuracy.

3.2. Truth Discovery

We will use the same truth discovery method as PPFDL [7] to compute the aggregation weights of different users' data. The aggregation weight w_i for user u_i is determined as follows:

$$w_i = \frac{\mathcal{C}}{F\left(\boldsymbol{x}_i^T, \boldsymbol{x}^{T-1}\right)}, \tag{1}$$

where $\mathcal{C}$ is the coefficient used to scale the user weights, $F\left(\boldsymbol{x}_i^T, \boldsymbol{x}^{T-1}\right)$ measures the distance between user u_i's local gradient $\boldsymbol{x}_i^T$ and the global gradient $\boldsymbol{x}^{T-1}$.

A shorter distance between $\boldsymbol{x}_i^T$ and $\boldsymbol{x}^{T-1}$ implies higher data quality for that user, and thus a larger weight w_i. As mentioned in the literature [25], user gradient $\boldsymbol{x}_i^T$ with smaller distances can be assigned larger aggregation weights w_i, which ensures that high-quality data mainly contribute to the global model.

After calculating the aggregation weights for all users, the global gradient is updated as follows.

$$\boldsymbol{x}^T = \frac{\sum_{i=1}^{n} w_i \boldsymbol{x}_i^T}{\sum_{i=1}^{n} w_i}. \tag{2}$$

3.3. Secret Sharing

Shamir secret sharing is a seminal threshold scheme that partitions a secret into multiple shares. The scheme comprises two core routines: secret distribution $SS.sh\left(\cdot\right)$ and secret reconstruction $SS.re\left(\cdot\right)$. The specific algorithms are as follows:

- $\left\{\left(\beta_i, f\left(\beta_i\right)\right)\right\}_{u_i \in U} \leftarrow SS.sh\left(s, U, t\right)$: given a secret s, a set $U = \left\{u_i\right\}_{i\in[1,n]}$ of all users, and a threshold t. Randomly choose $t-1$ coefficients

$a_i \in \mathbb{Z}_q$ to define a polynomial

$$f(x) = s + a_1 x + a_2 x + \cdots + a_{t-1} x^{t-1} \bmod q. \tag{3}$$

The share of the secret s can be denoted as $\{(\beta_i, f(\beta_i))\}_{u_i \in U}$.

- $s \leftarrow SS.re\left(\{(\beta_i, f(\beta_i))\}_{u_i \in U}, t\right)$: given any t shares $(\beta_i, f(\beta_i))$, the secret s can be reconstructed utilizing the Lagrange interpolation method. The following formula can efficiently recover secrets

$$s = \sum_{j=0}^{t} f(\beta_j) \prod_{m \neq j}^{t} \frac{\beta_m}{\beta_m - \beta_j} \bmod q. \tag{4}$$

The Shamir secret sharing has additive homomorphism [26]. Assuming two users u_1 and u_2 respectively hold secrets s and s', user u_1 computes the shares set of s as $\{(\beta_i, f(\beta_i))\}_{u_i \in U}$ by $SS.sh(s, U, t)$, and user u_2 utilizes $SS.sh(s', U, t)$ to calculate the set of shares of s' to be $\{(\beta_i, f'(\beta_i))\}_{u_i \in U}$. Then the secret $s + s'$ can be reconstructed by:

$$s + s' \leftarrow SS.re(\{(\beta_i, f(\beta_i) + f'(\beta_i))\}_{u_i \in U}, t). \tag{5}$$

Shamir secret sharing is linear over the field F. Local addition of shares gives a share of the sum, and reconstruction returns the exact value in F. Therefore, repeated additions of shares do not accumulate noise; the interpolation only changes the underlying polynomial accordingly.

3.4. Diffie-Hellman Key Agreement

The Diffie-Hellman protocol enables two parties to establish a common secret key across a public channel without any prior exchange of confidential data. The resulting shared secret key can be used for subsequent encrypted communications. It involves three types of probabilistic polynomial-time algorithms:

- $pp \leftarrow KA.setup(1^\lambda)$: given the security parameter λ, this routine outputs the public parameter pp.
- $(sk_u, pk_u) \leftarrow KA.gen(pp)$: using pp, the algorithm produces the user's private and public keys (sk_u, pk_u).
- $s_{u,v} \leftarrow KA.agree(sk_u, pk_v)$: combining user u's private key sk_u with user v's public key pk_v, the procedure derives the shared secret $s_{u,v}$.

Note that, $s_{u,v} = KA.agree(sk_u, pk_v)$ is equal to $s_{v,u} = KA.agree(sk_v, pk_u)$.

3.5. Symmetric Encryption

In symmetric cryptography, the same key serves for both encryption and decryption. A symmetric encryption system consists of three algorithms:

- KeyGen(λ) $\rightarrow k$: provided with security parameter λ, KeyGen returns a symmetric key k.
- Enc(k, m) $\rightarrow c$: given key k and plaintext m, Enc produces the ciphertext c.
- Dec(k, c) $\rightarrow m$: supplied with key k and ciphertext c, Dec recovers the plaintext m.

A symmetric encryption system satisfies the following correctness property: $Dec(k, Enc(k, m)) = m$.

4. Problem Statement

This section presents the system architecture and the associated threat assumptions of our scheme.

4.1. System Model

As shown in Fig.1, our system comprises three entities: trusted authority, cloud server, and users.

- **Trusted Authority (TA):** The TA acts as an honest third party, tasked with parameter initialization and per-user key-pair generation. It delivers to each user u_i a unique key pair (pk_i, sk_i) over a secure channel, keeps no copy of any sk_i, and holds no protocol key of its own after setup.
- **Cloud Server (CS):** The CS has sufficient computing resources but no training dataset. It updates the global model by combining users' gradients according to their assigned weights. It also compiles users' verification labels and returns both the aggregated result and these labels to each user.

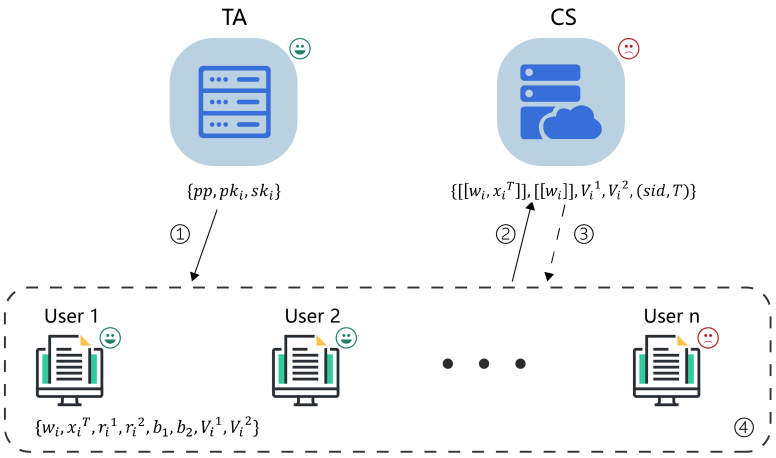

Figure 1: System Model (Legend: TA securely distributes pp and (pk_i, sk_i), then goes offline; pp means $\{\boldsymbol{x}^0, \boldsymbol{a}, B, c, sid\}$. Solid arrows mean uploads to server; dashed arrows mean server-relayed encrypted peer shares; dotted box means user trust boundary. $[\![\cdot]\!]$ means masked values; V_i^1, V_i^2 mean per-round verification tags. Numbers 14 mark the four phases: setup, local computation and upload, secure aggregation, verification and update. smile icon means benign and frown icon means malicious.)

- **Users (clients; u_i):** Each user holds a private local dataset. Some users' data are high quality, whereas others are low quality. Each user trains locally to obtain its gradient. All users share a private authenticated channel with the CS. The encrypted gradients, encrypted weights, and verification labels are then sent to the CS over this channel. After aggregation, users receive the aggregate and the labels for integrity checking.

Abbreviations. We use TA for the trusted authority, CS for the cloud server, and u_i for user i throughout.

4.2. Threat Model

Following the VerSA [8] threat model, the TA is fully trusted and never colludes with any participant. The CS is malicious, who attempts to access users' private data. Furthermore, it could manipulate the aggregation results. All users are assumed to be honest-but-curious: they follow the protocol faithfully yet may attempt to infer other parties' private information. In addition, the users will not collude with the CS.

We target cross-device federated learning where the server may be untrusted. Clients are honest but curious and may hold low quality data; they follow the protocol and do not craft adversarial updates.Our design focuses on server-side integrity and privacy via verifiable secure aggregation.

5. The Proposed Scheme

This section first briefly describes the process of the proposed scheme, and then provides a detailed description of the scheme.

5.1. Overview

Our proposed scheme has four phases: setup phase, local computation phase, secure aggregation phase, and gradient update phase.

In the setup phase, TA generates the system parameters pp and the key pair for each user (pk_i, sk_i), TA send pp, pk_i, sk_i to user u_i via the secure channel and does not retain any user's secret key. After delivery, TA goes offline. User u_i uploads pk_i to the CS. The CS collects the received public keys and broadcasts them to the online users U_1. To avoid seed reuse, a master seed B is included in pp For the T-th round, parties derive a fresh per-round seed $b^{(T)} = PRF(B, sid||T)$, where PRF denotes a pseudorandom function, to generate one-time masks and verification labels, where sid is the session identifier.

In the local computation phase, user in U_1 obtains his local gradient by local model training. Then users calculate the weights of their own gradients using equation (1). Next, user randomly selects two masks to encrypt the weighted gradient and weight respectively. Meanwhile, user splits the two masks by secret sharing. In addition, user computes the verification labels of weighted gradient and weight. Finally, user sends above message to the CS.

In the secure aggregation phase, the CS leverages the additive homomorphism of Shamir secret sharing to sum the weighted gradients and the corresponding weights. Because these computations are exact over the field F, they introduce no numerical noise; the outputs remain valid Shamir shares of the plaintext sums. The server also sums the users' verification labels so that each user can check the correctness of the aggregates. It then sends the aggregated values and the proofs to the online users.

In the gradient update phase, the online users can verify the results of the aggregated weighted gradient and the aggregated weights. If validation is

successful, the users will adopt the aggregated results and update the global gradient. Otherwise, they will abort the computation.

5.2. The Proposed Scheme

Let $\mathbb{F}$ be a field, n be an integer number, m be the dimension of gradient, and t be the threshold of secret sharing. The four phases mentioned above are described in detail as follows.

Phase 1: Setup phase

There are n users $U = \{u_i, i \in [1, n]\}$. The TA samples a public session identifier sid and generates the system parameters $pp = \{\boldsymbol{x}^0, \boldsymbol{a}, B, c, sid\}$, where $\boldsymbol{x}^0 \in \mathbb{F}^m$ is the initial gradient, $\boldsymbol{a} \in \mathbb{F}^m$ is a random vector, $B \in \mathbb{F}$ is the master seed, and $c \in \mathbb{F}$ are random numbers. For round T, parties derive a fresh seed $b^{(T)} = \mathrm{PRF}(B, sid \| T)$ to generate one-time masks and verification labels; no seed is reused across rounds.

For each user u_i, the TA runs the key-generation algorithm to produce a unique Diffie-Hellman key pair (pk_i, sk_i) that will be used to protect pairwise shares between users. The TA delivers (pk_i, sk_i) and pp to u_i over a secure channel, erases sk_i immediately after delivery, and retains no copy. The TA holds no protocol key of its own and goes offline after setup. User u_i uploads pk_i to the CS. Let $U_1 \subseteq U$ be the set of users that submitted public keys; the CS broadcasts these public keys to all users in U_1.

For $T = 1$, users ignore any server-supplied initial model and use $\boldsymbol{x}^0$ embedded in pp. The identifier sid labels the training session and is included in all messages and PRF inputs to bind rounds and prevent cross-session replay.

Phase 2: Local computation phase

At round T, we assume that users have gained the $(T-1)$-th global gradient $\boldsymbol{x}^{T-1}$ from the CS.

Firstly, each user $u_i \in U_1$ obtains the gradient $\boldsymbol{x}_i^T$ by local model training. According to truth discovery technology, user u_i computes the weight of this round as follows:

$$w_i = \frac{\chi^2_{\left(1-\frac{\alpha}{2}\right),m}}{\left(\boldsymbol{x}_i^T - \boldsymbol{x}^{T-1}\right)^2}, \tag{6}$$

where χ^2 represents the chi-squared distribution, and α denotes the corresponding significance level. As illustrated in [27], local gradients trained

with high-quality data are closer to the global gradient than those trained with low-quality data. As shown in the above formula (6), high-quality data are assigned a higher weight, while low-quality data receive a lower weight. Therefore, the gradients aggregated by the CS will be more biased towards those from high-quality data, thereby reducing the negative impact of low-quality data on the global model. Therefore, the χ^2-based reliability weight in (6) concretely instantiates the truth-discovery idea introduced in Sec. 3.2.

Secondly, user u_i randomly selectes a vector $\boldsymbol{r}_i^1 \in \mathbb{F}^m$ and a value r_i^2 as mask to encrypt the weighted gradient $w_i \boldsymbol{x}_i^T$ and the weight w_i:

$$\llbracket w_i \boldsymbol{x}_i^T \rrbracket = w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1 + \boldsymbol{b}_1, \tag{7}$$

$$\llbracket w_i \rrbracket = w_i + r_i^2 + b_2, \tag{8}$$

where $\boldsymbol{b}_1 \in \mathbb{F}^m$ and $b_2 \in \mathbb{F}$ are derived from the per-round seed $b^{(T)} = PRF(B, sid||T)$ using domain separation, e.g., $\boldsymbol{b}_1 = PRF(b^{(T)}, \texttt{"mask:x"})$ and $b_2 = PRF(b^{(T)}, \texttt{"mask:w"})$. Here, we employ a two-layer masking method to encrypt the weighted gradient. The first-layer mask $\boldsymbol{r}_i^1$ is used to hide the weighted gradient, while the second-layer mask $\boldsymbol{b}_1$ serves to protect the random number $\boldsymbol{a}$ within the subsequent verification tag $\boldsymbol{V}_i^1$. Similarly, the weight w_i is encrypted using the same idea.

Meanwhile, user u_i uses secret sharing to split the mask $\boldsymbol{r}_i^1$ and r_i^2, i.e.

$$(u_j, \boldsymbol{r}_{i,j}^1)_{u_j \in U_1} \leftarrow SS.sh(\boldsymbol{r}_i^1, |U_1|, t), \tag{9}$$

$$(u_j, r_{i,j}^2)_{u_j \in U_1} \leftarrow SS.sh(r_i^2, |U_1|, t). \tag{10}$$

Then user u_i uses symmetric encryption to encrypt the secret share:

$$\boldsymbol{e}_{i,j} = Enc\left(KA.agree\left(sk_i, pk_j\right), \boldsymbol{r}_{i,j}^1 || r_{i,j}^2\right), \tag{11}$$

where $Enc(\cdot)$ refers to the symmetric encryption algorithm, and $KA.agree(\cdot)$ refers to the key agreement algorithm.

In addition, user u_i generates the verification labels of weighted gradient $w_i \boldsymbol{x}_i^T$ and weight w_i as follows:

$$\boldsymbol{V}_i^1 = \boldsymbol{a} \circ (w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1) \tag{12}$$

$$V_i^2 = c(w_i + r_i^2) \tag{13}$$

where " $\circ$ " is the Hadamard product [28].

Finally, user u_i sends $(\{u_j, \boldsymbol{e}_{i,j}\}_{u_j \in U_1}, [\![w_i \boldsymbol{x}_i^T]\!], [\![w_i]\!], \boldsymbol{V}_i^1, V_i^2)$ to the CS. The set of all users who send above message to the CS is denoted as U_2. Note that $U_2 \subset U_1$. The CS distributes $\{\boldsymbol{e}_{i,j}\}_{u_i \in U_2}$ to the corresponding users u_j in U_2.

Phase 3: Secure aggregation phase

User u_j receives $\{\boldsymbol{e}_{i,j}\}_{u_i \in U_2}$ from the CS, and decrypts them to obtain the corresponding mask shares:

$$\boldsymbol{r}_{i,j}^1 || r_{i,j}^2 = Dec(KA.agree(sk_j, pk_i), \boldsymbol{e}_{i,j}), \tag{14}$$

where $Dec(\cdot)$ refers to the symmetric decryption algorithm.

Then, user u_j calculates $\boldsymbol{R}_j^1 = \sum_{u_i \in U_2} \boldsymbol{r}_{i,j}^1$ and $R_j^2 = \sum_{u_i \in U_2} r_{i,j}^2$, and sends $(\boldsymbol{R}_j^1, R_j^2)$ to the CS. The set of all users who send above message to the CS is denoted as U_3. Note that $U_3 \subset U_2$. The CS utilizes the homomorphism of the secret sharing to reconstruct the sum of masks $\boldsymbol{r}_i^1$ and r_i^2 as follows:

$$\boldsymbol{R}^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{r}_i^1 = SS.re(\{(u_j, \boldsymbol{R}_j^1)\}_{u_j \in U_3}, t), \tag{15}$$

$$R^2 = \sum\nolimits_{u_i \in U_2} r_i^2 = SS.re(\{(u_j, R_j^2)\}_{u_j \in U_3}, t). \tag{16}$$

Meanwhile, the CS aggregates the weighted gradient and weight as follows:

$$\boldsymbol{X}_{agg} = \sum\nolimits_{u_i \in U_2} [\![w_i \boldsymbol{x}_i^T]\!] - \sum\nolimits_{u_i \in U_2} \boldsymbol{r}_i^1 = \sum\nolimits_{u_i \in U_2} w_i \boldsymbol{x}_i^T + |U_2| \boldsymbol{b}_1 \tag{17}$$

$$W_{agg} = \sum\nolimits_{u_i \in U_2} [\![w_i]\!] - \sum\nolimits_{u_i \in U_2} r_i^2 = \sum\nolimits_{u_i \in U_2} w_i + |U_2| b_2 \tag{18}$$

In addition, the CS aggregates the verification labels of all users in U_2 as follows:

$$\boldsymbol{V}_{agg}^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{V}_i^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{a} \circ (w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1) \tag{19}$$

$$V_{agg}^2 = \sum\nolimits_{u_i \in U_2} V_i^2 = \sum\nolimits_{u_i \in U_2} c(w_i + r_i^2) \tag{20}$$

Finally, the CS sends the aggregation results ($\boldsymbol{X}_{agg}$, W_{agg}), verification labels ($\boldsymbol{V}^1_{agg}, V^2_{agg}$), and aggregation masks ($\boldsymbol{R}^1$, R^2) to all users.

Phase 4: Gradient update phase

Users first check that the tuple (sid, T) matches the expected round and has not been seen before, duplicates are rejected. After the users receive above message, they firstly verify the correctness of the aggregation results. That is, users check whether the following two equations hold:

$$\boldsymbol{V}^1_{agg} = \boldsymbol{a} \circ (\boldsymbol{X}_{agg} - |U_2|\boldsymbol{b}_1 + \boldsymbol{R}^1) \quad (21)$$

$$V^2_{agg} = c(W_{agg} - |U_2|b_2 + R^2) \quad (22)$$

If above two equations hold, users update the global gradient as $\boldsymbol{x}^{T+1}$ as follows:

$$\boldsymbol{x}^{T+1} = \frac{\boldsymbol{X}_{agg} - |U_2|\boldsymbol{b}_1}{W_{agg} - |U_2|b_2} = \frac{\sum_{u_i \in U_2} w_i \boldsymbol{x}_i^T}{\sum_{u_i \in U_2} w_i}. \quad (23)$$

If the verification fails, abort.

Notably, if the online users are fewer than the threshold t in any time, the scheme aborts. That is, we need $|U| \geq |U_1| \geq |U_2| \geq |U_3| \geq t$. From another perspective, the proposed scheme supports user dropouts. As long as the number of online users in each phase exceeds the threshold t, the CS can correctly aggregate the weighted gradients and weights.

6. Security Analysis

In this section, We describe two games. In the privacy game, the adversary controls the CS and any coalition of fewer than t clients. The system uses a Shamir threshold sharing scheme and the adversary observes the internal state of corrupted parties and all protocol messages. The task is to distinguish a real execution from an ideal execution that reveals only the two aggregates $\sum_i w_i x_i^T$ and $\sum_i w_i$. In the verification game, the adversary is a malicious server that tries to make honest clients accept an incorrect aggregate. The next theorems state indistinguishability for Priv and rejection of forgeries for Ver under the stated assumptions.

6.1. Data Privacy

Theorem 1 (Computational privacy under PRF security). *The system uses a Shamir threshold scheme with threshold t and n total clients over a finite field $\mathbb{F}$. The trusted authority delivers per-user keys over secure channels. In each round the masks b_1 and b_2 are derived from a pseudorandom function $\mathsf{PRF}[B, \cdot]$ keyed by a master seed B. The adversary controls the server and any coalition of fewer than t clients. Under these conditions the adversarys view in our protocol is computationally indistinguishable from an ideal view that reveals only $\sum_i w_i x_i^T$ and $\sum_i w_i$. If the outputs of the pseudorandom function are replaced by truly uniform values then the privacy becomes statistical.*

Proof: Use a hybrid argument that replaces the outputs of the pseudorandom function by uniform values. Any distinguisher gives a distinguisher for the pseudorandom function. With fewer than t shares the Shamir scheme hides the remaining contributions, so the view depends only on the two released aggregates. This yields computational privacy and yields statistical privacy in the uniform hybrid.

6.2. Verifiability

Theorem 2 (Soundness of aggregation verification). *We keep the setting of Theorem 1. The vector $a \in \mathbb{F}^m$ and the scalar $c \in \mathbb{F}$ are user-held secrets that are included in the public parameters. The masks b_1 and b_2 are derived per round from the master seed B using PRF. Let the server output X_{agg} and W_{agg}. If this output is not equal to $\sum_i w_i x_i^T + |U_2|\, b_1$ and $\sum_i w_i + |U_2|\, b_2$ then the probability that all honest clients accept is negligible in the security parameter. Otherwise a successful adversary yields either a tag forger that succeeds without knowledge of a and c, or a distinguisher for the pseudorandom function.*

Proof: Use a reduction. If an adversary makes honest clients accept an incorrect aggregate then we can either recover tags that match the unknown secrets a and c and obtain a forgery or use the success probability to distinguish real outputs of PRF from uniform. The round mask b_1 is unknown to the server, so correlating V_{agg}^1 with $X_{\text{agg}} - R_1$ cannot reveal a. The factor $|U_2|\, b_1$ blocks such inference. Any non-negligible success therefore contradicts tag security or PRF security.

6.3. Attack Analysis and Countermeasures

Collusion between the server and clients. Privacy holds against any coalition of fewer than t users even when the server is malicious, because fewer than t Shamir shares are independent of the secret. If $\geq t$ users collude with the server, privacy can be broken. Hence t should be set no smaller than the minimal cohort size we are willing to reveal at round time, and the protocol aborts whenever $|U_3| < t$. In practice we also require $|U_2| \geq k_{\min}$ $(\geq t)$ before releasing any aggregate.

Replay or tag-reuse by the server. Every ciphertext, tag, and mask is bound to (sid, T), and per-round seeds are fresh. Clients check that the received pair (sid, T) matches the expected round and reject duplicate or out-of-order tuples. Replaying an older $(\boldsymbol{X}_{agg}, W_{agg}, \boldsymbol{V}^1_{agg}, V^2_{agg})$ from round T' cannot pass the checks for $T \neq T'$, because the masks and tags are derived from the new seed $b^{(T)}$ and thus differ.

Leakage when few participants remain. Aggregates computed from very small cohorts can leak via differencing. We therefore enforce a minimum cohort size and weight dispersion policy: if $|U_2| < k_{\min}$ or if $\max_i w_i$ is too large relative to the total, the round aborts. Concretely, for a chosen $0 < \rho < 1$, require

$$\max_{u_i \in U_2} w_i \;\leq\; \rho \cdot \sum_{u_i \in U_2} w_i \quad \text{and} \quad |U_2| \geq k_{\min}\ (\geq t).$$

This prevents any single user (or tiny set) from dominating the weighted sum.

Reuse of random seeds across rounds. To avoid cross-round correlation, per-round values are derived with domain separation:

$$b^{(T)} = PRF(B,\, sid \,||\, T), \boldsymbol{b}_1 = PRF(b^{(T)},\, \texttt{"mask:x"}), b_2 = PRF(b^{(T)},\, \texttt{"mask:w"}).$$

No seed is reused across rounds or purposes. If a client restarts, it recomputes the same $b^{(T)}$ from (B, sid, T) and continues; if the session restarts, a fresh sid is chosen.

7. Experiments

This section first describes the computing environment and cryptographic setup, then the datasets and partitioning strategy, then the simulation of low-quality data, and finally the evaluation protocol used in the following subsections on functionality, accuracy, and efficiency.

7.1. Datasets and Partitioning

All simulations run on a Windows 11 workstation with an Intel Core i5-9300H at 2.40 GHz, 16 GB RAM, and an NVIDIA GeForce GTX 1650 GPU. The cryptographic layer uses elliptic curve DiffieHellman and a Shamir secret sharing scheme with threshold t out of n implemented in Python.

We evaluate three image classification corpora. MNIST contains handwritten digits with sixty thousand training samples and ten thousand test samples. CIFAR-10 contains natural color images of size thirty two by thirty two in ten classes with fifty thousand training images and ten thousand test images. FEMNIST groups characters by writer so one client maps to one writer.

Data are partitioned to cover both idealized and realistic federated regimes. MNIST is split to one hundred clients under an independent and identically distributed assignment. CIFAR-10 is split to one hundred clients with a non-IID Dirichlet label allocation where the concentration alpha takes 0.1, 0.5, and 10 to control skewness. FEMNIST uses its native writer as client split which is non-IID by construction. Unless stated otherwise, each round samples one hundred clients, each client performs one local epoch with learning rate 0.01 and batch size 10, the number of rounds is 100, and the secret sharing threshold is t equal to 0.8N so that up to twenty percent drop out does not abort a round. MNIST and FEMNIST use a four convolution two fully connected network, and CIFAR-10 uses the same backbone so that accuracy differences arise from data and partitioning rather than model capacity.

7.2. Simulation of low-quality data

Low quality means non adversarial defects that reduce utility while clients still follow the protocol. We vary their prevalence across clients and keep all other training and cryptographic settings unchanged.

- **Label noise.** Symmetric noise flips labels at rates 0.1, 0.2, and 0.3 to any other class with equal probability. Asymmetric noise flips labels only within confusable classes under the same rates. For CIFAR-10 we use standard pairs such as truck to automobile, bird to airplane, deer to horse, and cat to dog. For FEMNIST the flips stay within digit groups or within letter groups.

- **Input perturbations on local training data.** For designated clients we degrade local training images before local SGD using simple operators that reflect sensor or pipeline imperfections. Gaussian noise with

standard deviation 0.05 and 0.10 on images normalized to zero to one. Gaussian blur with kernel size three and five. Brightness and contrast jitter with factors 0.8 and 1.2. The server observes only masked updates.

- **Imbalance and heterogeneous client size.** We construct long tailed class frequencies where the largest class has ten times, fifty times, or one hundred times as many samples as the smallest class. We then assign samples to clients so that some clients hold only a small local dataset while others hold more. All other settings remain unchanged.

- **Partial occlusion.** We apply random erasing to a client local samples with erase probability 0.1 and 0.2 and with square patches whose side length occupies 0.2 and 0.4 of the image edge.

We let p denote the fraction of clients carrying the defect. We use $p \in \{0, 0.05, 0.10, 0.20, 0.30\}$. In each round, among the participating clients, a proportion p are designated as low-quality. All other training and cryptographic parameters remain identical to the baseline so that differences reflect robustness to data quality alone.

7.3. Functionality

We compare the functionality of representative designs in the same order as Table 1. SecAgg [29] protects per client updates with double masking and Shamir secret sharing and remains tolerant to client dropouts, and it does not provide end user verification of the server aggregate. SecProbe [6] perturbs the training objective with a differential privacy functional mechanism and selects participants with the differential privacy exponential mechanism so that a participants quality is hidden from others, and it does not add an explicit mechanism for dropout tolerance. PPFDL [7] computes truth discovery weights over encrypted data using additively homomorphic encryption such as Paillier for sums and Yaos garbled circuits for division and related operations, and it assumes two noncolluding servers and describes robustness when some users drop out. ESFL [14] performs privacy preserving weighted averaging and gives each client a verification tag that enables a local check of the returned weighted sum, and it does not target data quality aware reweighting. VeriFL [10] achieves verifiable aggregation with a linearly homomorphic hash and commitments and uses verification communication that

does not grow with the model dimension, and it composes with secure aggregation while it does not incorporate data quality weighting. In contrast, our scheme preserves privacy, enables client side verification, is robust to low quality updates and to client dropouts, and runs with a single server.

Table 1: Functional comparison with previous schemes (DP = Differential Privacy; Tag = one-time validation tag; LQ-Robust = robustness to low-quality clients; Drop-out Tol. = tolerance to client drop-out)

Scheme	Privacy-Preserving	Verifiable	LQ-Robust	Drop-out Tol.	Single-server	Technique
SecAgg [29]	✓	×	×	✓	✓	Double-mask/ Shamir
SecProbe [6]	✓	×	✓	×	✓	DP objective perturbation/ DP exponential mechanism
PPFDL [7]	✓	×	✓	✓	×	HE (Paillier)/ Garbled Circuits / Truth Discovery
ESFL [14]	✓	✓	×	×	✓	Masking/ Verification Tag (weighted avg.)
VeriFL [10]	✓	✓	×	×	✓	Linearly homomorphic hash / Commitments
Ours	✓	✓	✓	✓	✓	Double-mask/ Secret Sharing / Truth Discovery

7.4. Accuracy

To evaluate robustness against low-quality clients, we deliberately compare our method with only ESFL [14] and PPFDL [7]—the two baselines that share the same secret-sharing, single-round aggregation backbone but differ precisely in their handling of unreliable data. ESFL keeps every client equally weighted and thus represents the no-robustness reference, whereas PPFDL applies the same χ^2-based weighting rule as ours but relies on a Paillier homomorphic-encryption protocol executed jointly by two non-colluding servers. We omit SecAgg and SecProbe here because neither scheme applies any weighting; their accuracy would overlap with ESFL when $P = 0$ and drop even faster once noisy clients are introduced.

All subsequent experiments use MNIST, a 4-Conv-2-FC CNN, 100 clients, 100 rounds, and a learning rate of 0.01. We randomly flip the labels of P% of clients to mimic low-quality data, where $P \in 0, 0.05 \ldots, 0.30$, each configuration is repeated five times, and Fig. 2 plots the mean accuracy with one-standard-deviation error bars. When $P = 0$ all three methods converge to approximately 0.98. As P rises, ESFL plummets to 0.517 at $P = 0.30$ because unweighted noisy clients dominate the update. By contrast, PPFDL and our scheme remain above 0.89; our single-server variant even achieves 0.90 without discarding any round.

These results show that χ^2 weighting is essential for noisy environments; it lifts accuracy by more than 37 percentage points over ESFL at $P = 0.30$. Moreover, our design matches PPFDL-level robustness while avoiding its two-server requirement and large Paillier ciphertexts, thus providing the best accuracy-to-deployment trade-off among the compared approaches.

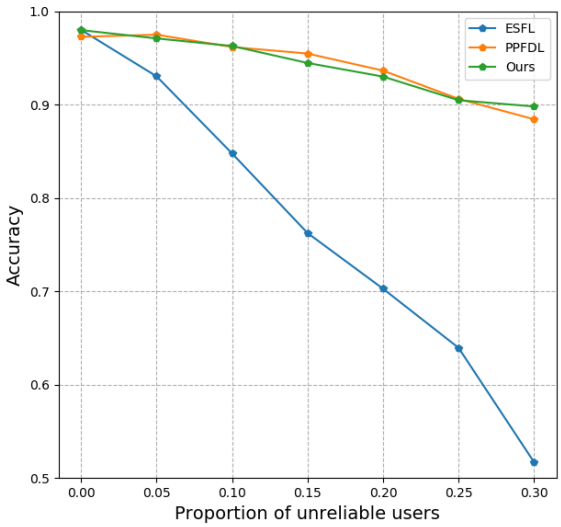

Figure 2: Test accuracy versus the proportion P of low-quality clients. (Each point is averaged over five independent runs; error bars indicate one standard deviation).

7.5. Efficiency

To measure the cost introduced solely by verifiability, we reuse the MNIST setting of Sec.7.2, 4-Conv-2-FC CNN, 100–500 clients, 100 rounds, learning rate 0.01, batch size 10but switch off the χ^2-weighting module. Two single-server baselines are retained for comparison: ESFL [14] (one-time tag) and VeriFL [10] (Bulletproof proof). SecAgg and SecProbe are omitted here because they contain no verification layer, so their traffic equals the plaintext baseline. For each N we record (i) per-client uplink bytes and (ii) server CPU time per round.

Figure 3 shows that our scheme and ESFL overlap at approximately 0.95 MB when $N = 100$ and 0.98 MB when $N = 500$; the extra cost is merely a 16-byte tag, and server latency stays below 5 ms. By contrast, VeriFL carries an additional approximately 640 kB, raising traffic to 1.6-1.7 MB and increasing proof-generation time from 0.12 s (100 clients) to 0.40 s (500 clients).

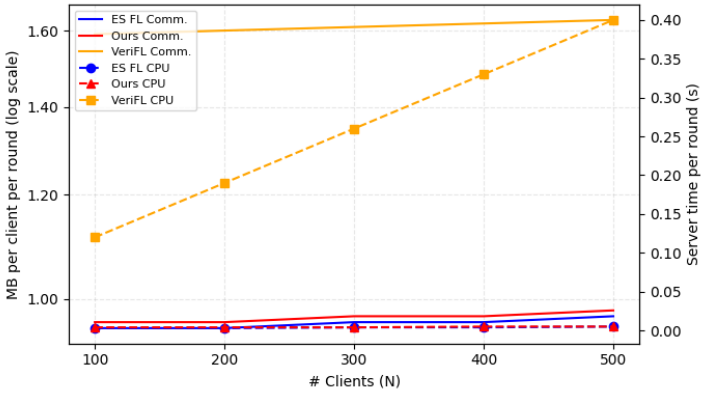

Figure 3: Per-Round Communication Cost and Server CPU Overhead

Hence, with the same bandwidth and latency as ESFL, our design removes ESFL's robustness gap (Sec.7.2) while avoiding VeriFL's heavy zero-knowledge burden. When the χ^2 weights are re-enabled, the identical communication budget yields markedly higher accuracy under noisy clients, giving our scheme the best trade-off among all evaluated single-server methods.

8. Limitations and Future Work

Our scheme targets an untrusted server with honest but curious clients who may upload low-quality updates caused by noisy or imbalanced data. The protocol does not defend against crafted poisoning or backdoor updates. The threshold sharing layer requires at least t active users in each phase. When participation drops below t the round aborts. Adaptive thresholds and late share recovery are promising extensions. The trusted authority performs setup and distributes per-user keys. Moving to user-side key generation or a distributed setup would further reduce trust. Our verification checks the algebraic correctness of sums and tags, not the semantic quality of client updates. Extending privacy-preserving aggregation with robust aggregation or admission checks is a realistic direction, because many robust rules rely on access to individual updates that secure aggregation hides. We encode real-valued updates as fixed-point integers over a finite field and use exact field arithmetic. A systematic study of scaling, clipping, and modulus selection to avoid wrap-around under different models and datasets is practical future work.

9. Conclusion

We presented a verifiable secure aggregation scheme for federated learning with an untrusted cloud server and honest but curious clients who may hold low-quality data. The protocol combines truth discovery with masked secret sharing so that higher-quality updates receive larger weights while the server only observes protected aggregates. Two-layer masking and per-round tags allow each online user to check the returned result and prevent the server from passing a falsified aggregation. The security analysis shows that privacy holds when the number of colluding parties is fewer than t, and the aggregation remains exact due to the linearity of Shamir secret sharing. Experiments on standard image benchmarks indicate that weighting non-adversarial low-quality clients improves accuracy while keeping the communication cost comparable to single-server baselines.

The design matches practical cross-device deployments with a single orchestration server and lightweight per-round checks. It enables privacy-preserving aggregation and end-user verifiability without exposing individual updates.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical Statement

No conflict of interest exists in the submission of this manuscript, and the manuscript is approved by all authors for publication. We would like to declare that the work described was original research that has not been published previously, and is not under consideration for publication elsewhere, in whole or in part.

Acknowledgments

We acknowledge the partial support of this research by the National Natural Science Foundation of China under Grant 12401663.

References

[1] Michelle Goddard. The EU General Data Protection Regulation (GDPR): European regulation that has a global impact. *International Journal of Market Research*, 59(6):703–705, 2017.

[2] H Brendan McMahan, FX Yu, P Richtarik, AT Suresh, D Bacon, et al. Federated learning: Strategies for improving communication efficiency. In *Proceedings of the 29th Conference on Neural Information Processing Systems (NIPS), Barcelona, Spain*, pages 5–10, 2016.

[3] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2017.

[4] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1322–1333, 2015.

[5] Yiran Li, Hongwei Li, Guowen Xu, Xiaoming Huang, and Rongxing Lu. Efficient privacy-preserving federated learning with unreliable users. *IEEE Internet of Things Journal*, 9(13):11590–11603, 2022.

[6] Lingchen Zhao, Qian Wang, Qin Zou, Yan Zhang, and Yanjiao Chen. Privacy-preserving collaborative deep learning with unreliable participants. *IEEE Transactions on Information Forensics and Security*, 15:1486–1500, 2020.

[7] Guowen Xu, Hongwei Li, Yun Zhang, Shengmin Xu, Jianting Ning, and Robert H. Deng. Privacy-preserving federated deep learning with irregular users. *IEEE Transactions on Dependable and Secure Computing*, 19(2):1364–1381, 2022.

[8] Changhee Hahn, Hodong Kim, Minjae Kim, and Junbeom Hur. Versa: Verifiable secure aggregation for cross-device federated learning. *IEEE Transactions on Dependable and Secure Computing*, 20(1):36–52, 2023.

[9] Guowen Xu, Hongwei Li, Sen Liu, Kan Yang, and Xiaodong Lin. Verifynet: Secure and verifiable federated learning. *IEEE Transactions on Information Forensics and Security*, 15:911–926, 2020.

[10] Xiaojie Guo, Zheli Liu, Jin Li, Jiqiang Gao, Boyu Hou, Changyu Dong, and Thar Baker. Verifl: Communication-efficient and fast verifiable aggregation for federated learning. *IEEE Transactions on Information Forensics and Security*, 16:1736–1751, 2021.

[11] Anmin Fu, Xianglong Zhang, Naixue Xiong, Yansong Gao, Huaqun Wang, and Jing Zhang. Vfl: A verifiable federated learning with privacy-preserving for big data in industrial iot. *IEEE Transactions on Industrial Informatics*, 18(5):3316–3326, 2022.

[12] Yuanjun Xia, Yining Liu, Shi Dong, Meng Li, and Cheng Guo. Svca: Secure and verifiable chained aggregation for privacy-preserving federated learning. *IEEE Internet of Things Journal*, 11(10):18351–18365, 2024.

[13] Xianglong Zhang, Anmin Fu, Huaqun Wang, Chunyi Zhou, and Zhenzhu Chen. A privacy-preserving and verifiable federated learning scheme. In *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, pages 1–6, 2020.

[14] Zhen Yang, Ming Zhou, Haiyang Yu, Richard O. Sinnott, and Huan Liu. Efficient and secure federated learning with verifiable weighted average aggregation. *IEEE Transactions on Network Science and Engineering*, 10(1):205–222, 2023.

[15] Giorgio Patrini, Alessandro Rozza, Aditya Krishna Menon, Richard Nock, and Lizhen Qu. Making deep neural networks robust to label noise. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[16] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. 2019.

[17] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification. 2019.

[18] Qiang Li, Yuncheng Diao, Qian Chen, and Bingsheng He. Federated learning on non-iid data silos: An experimental study. 2021.

[19] Sebastian Caldas, Peter Wu, Tian Li, Jakub Konečný, H. Brendan McMahan, Virginia Smith, and Ameet Talwalkar. Leaf: A benchmark for federated settings. 2018.

[20] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. 2017.

[21] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečnỳ, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML (MLSys) Conference*, 2019. Production system design for cross-device FL.

[22] Micah J. Sheller, Brandon Edwards, G. Anthony Reina, Jason Martin, Sarthak Pati, Aikaterini Kotrotsou, Mikhail Milchenko, Weilin Xu, Daniel Marcus, Rivka R. Colen, and Spyridon Bakas. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Scientific Reports*, 10(12598), 2020.

[23] Ittai Dayan, Holger R. Roth, Aoxiao Zhong, et al. Federated learning for predicting clinical outcomes in patients with covid-19. *Nature Medicine*, 27:1735–1743, 2021.

[24] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[25] Chenglin Miao, Wenjun Jiang, Lu Su, Yaliang Li, Suxin Guo, Zhan Qin, Houping Xiao, Jing Gao, and Kui Ren. Cloud-enabled privacy-preserving truth discovery in crowd sensing systems. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, pages 183–196, 2015.

[26] Oded Goldreich. Secure multi-party computation. *Manuscript. Preliminary version*, 78(110):1–108, 1998.

[27] Jieyu Xu, Hongwei Li, Jia Zeng, and Meng Hao. Efficient and privacy-preserving federated learning with irregular users. In *ICC 2022 - IEEE International Conference on Communications*, pages 534–539, 2022.

[28] Elizabeth Million. The hadamard product. *Course Notes*, 3(6):1–7, 2007.

[29] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.